

Supplementary code for the article “Model-comparison effect size indices for logistic and other generalized linear models: Computation and power analysis”

Supplementary code: generating a sample with a known population R-squared

Purpose

This file illustrates the calibrate-then-sample method used throughout the simulations reported in the paper, applied in turn to a logistic, a multinomial, and an ordinal (proportional-odds) model whose population R-squared is exactly known in advance. The Gauss-Hermite quadrature that makes this possible – computing the population R-squared exactly, for any candidate coefficient scale, before a single case is drawn – is carried out throughout by `statmod::gauss.quad.prob()`, which returns quadrature nodes and weights for a standard normal density (weights already summing to 1, so that for any function f , $E[f(X)] = \sum(w * f(X_{gh}))$). Each function below builds its quadrature grid by transforming that base grid through the Cholesky factor of `x_cov` and shifting it by `x_mean`, giving quadrature nodes for the actual (correlated, possibly non-centered) predictor distribution.

The code below is a stripped-down extract of the corresponding branch of `sample_by_r2()` from the `Rsimcity` package (<https://github.com/mcfanda/RsimCity>): input validation, caching, and the branch for targeting a single predictor’s eta-squared rather than the whole model’s R-squared have all been removed, so that only the logic essential to each case remains. All of those variations, their defaults, and their full implementations can be found in the package itself.

The method in three steps

1. Build a numerical-integration grid over the joint distribution of the k predictors, using Gauss-Hermite quadrature.
2. Find, by root-finding, the coefficient scale that makes the *population* R-squared – computed exactly on that grid – equal the requested target.
3. Only then draw an actual finite sample of size n from the calibrated population.

Logistic model

```
sample_logistic_r2 <- function(n, R2, k = 1, beta = rep(1, k),
                              intercept = 0, x_mean = rep(0, k), x_cov = diag(k),
                              order = 10, seed = NULL) {

  ## Gauss-Hermite quadrature grid over the k predictors
  ## using statmod::gauss.quad.prob(). It returns nodes/weights for a standard normal density,
  ## with weights already summing to 1, so that for any function f, E[f(X)] = sum(w * f(X_gh)).
  ## The grid is then transformed through the Cholesky factor of x_cov and shifted by x_mean, giving qu
  ## for the actual (correlated, possibly non-centered) predictor distribution.
  gh <- statmod::gauss.quad.prob(order, dist = "normal")
  g <- as.matrix(expand.grid(rep(list(seq_len(order)), k)))
  Z <- matrix(gh$nodes[g], ncol = k)
  w <- apply(matrix(gh$weights[g], ncol = k), 1, prod)
  Xgh <- sweep(Z %*% chol(x_cov), 2, x_mean, "+")
```

```

## Population deviance R-squared, as a function of a coefficient scale `a`
## beta = a * beta direction; the model's probability at every grid node gives the
## deviance-based R-squared exactly (no simulation is involved in this computation).
r2_of_a <- function(a) {
  eta <- intercept + drop(Xgh %*% (a * beta))
  prob <- stats::plogis(eta)

  p <- cbind(1 - prob, prob)
  p <- pmax(p, .Machine$double.eps)
  p0 <- colSums(p * w) # population marginal probabilities
  D1 <- -2 * sum(w * rowSums(p * log(p))) # model deviance
  D0 <- -2 * sum(p0 * log(p0)) # null-model deviance
  1 - D1 / D0
}

## ---- 3. Root-find the coefficient scale that produces the target R2 -----
upper <- 1
while (r2_of_a(upper) < R2 && upper < 1e6) upper <- upper * 2
a <- stats::uniroot(function(a) r2_of_a(a) - R2, c(0, upper))$root
beta_final <- a * beta

## ---- 4. Draw the actual finite sample from the calibrated population -----
if (!is.null(seed)) set.seed(seed)
X <- sweep(matrix(stats::rnorm(n * k), n, k) %*% chol(x_cov), 2, x_mean, "+")
colnames(X) <- paste0("x", seq_len(k))
eta <- intercept + drop(X %*% beta_final)
y <- stats::rbinom(n, 1, stats::plogis(eta))

data.frame(y = y, X)
}

```

Code

```

dat <- sample_logistic_r2(n = 2000, R2 = .10, seed = 1)
mod <- glm(y ~ x1, data = dat, family = binomial())
1 - mod$deviance / mod$null.deviance # sample deviance R2, close to the .10 target

```

Example: a single predictor, R-squared = .10

```
## [1] 0.09601619
```

```

Sigma <- matrix(c(1, .3, .2,
                  .3, 1, .4,
                  .2, .4, 1), nrow = 3)

dat3 <- sample_logistic_r2(n = 5000, R2 = .20, k = 3, x_cov = Sigma, seed = 1)
mod3 <- glm(y ~ x1 + x2 + x3, data = dat3, family = binomial())
1 - mod3$deviance / mod3$null.deviance # close to the .20 target

```

Example: three correlated predictors, R-squared = .20

```
## [1] 0.1916439
```

Multinomial model

The multinomial model's category probabilities are given by the softmax of the $y_1 - 1$ non-reference logits (category 1 is the reference), each with its own k -vector of slopes – so `beta` here is a k by $y_1 - 1$ matrix rather than a plain vector – and its own intercept. The population deviance R-squared is computed exactly on the same quadrature grid, from the resulting category-probability matrix.

```
sample_multinomial_r2 <- function(n, R2, k = 1, y1 = 3,
                                beta = matrix(1, nrow = k, ncol = y1 - 1),
                                intercept = rep(0, y1 - 1),
                                x_mean = rep(0, k), x_cov = diag(k),
                                order = 10, seed = NULL) {

  ## Gauss-Hermite quadrature grid over the k predictors (statmod::gauss.quad.prob()),
  ## transformed through the Cholesky factor of x_cov and shifted by x_mean.
  gh <- statmod::gauss.quad.prob(order, dist = "normal")
  g <- as.matrix(expand.grid(rep(list(seq_len(order)), k)))
  Z <- matrix(gh$nodes[g], ncol = k)
  w <- apply(matrix(gh$weights[g], ncol = k), 1, prod)
  Xgh <- sweep(Z %*% chol(x_cov), 2, x_mean, "+")

  ## Category probabilities: softmax of the y1 - 1 non-reference logits, with
  ## category 1 as the (implicit, zero-logit) reference category.
  softmax <- function(eta) {
    eta <- cbind(0, eta)
    eta <- eta - apply(eta, 1, max)      # numerically stable softmax
    p <- exp(eta)
    p / rowSums(p)
  }

  ## Population deviance R-squared, as a function of a coefficient scale `a`
  ## (beta = a * beta direction), computed exactly on the quadrature grid.
  r2_of_a <- function(a) {
    eta <- sweep(Xgh %*% (a * beta), 2, intercept, "+")
    p <- softmax(eta)
    p <- pmax(p, .Machine$double.eps); p <- p / rowSums(p)
    p0 <- colSums(p * w)                # population marginal probabilities
    D1 <- -2 * sum(w * rowSums(p * log(p))) # model deviance
    D0 <- -2 * sum(p0 * log(p0))         # null-model deviance
    1 - D1 / D0
  }

  ## ---- Root-find the coefficient scale that produces the target R2 -----
  upper <- 1
  while (r2_of_a(upper) < R2 && upper < 1e6) upper <- upper * 2
  a <- stats::uniroot(function(a) r2_of_a(a) - R2, c(0, upper))$root
  beta_final <- a * beta

  ## ---- Draw the actual finite sample from the calibrated population -----
  if (!is.null(seed)) set.seed(seed)
  X <- sweep(matrix(stats::rnorm(n * k), n, k) %*% chol(x_cov), 2, x_mean, "+")
  colnames(X) <- paste0("x", seq_len(k))
  eta <- sweep(X %*% beta_final, 2, intercept, "+")
}
```

```

prob <- softmax(eta)

u <- stats::runif(n) # draw a category per case
cum <- t(apply(prob, 1, cumsum))
y <- 1L + rowSums(u > cum)
y <- pmin(y, y1)

data.frame(y = factor(y, levels = seq_len(y1)), X)
}

```

Code

```

dat <- sample_multinomial_r2(n = 5000, R2 = .10, seed = 1)
full <- nnet::multinom(y ~ x1, data = dat, trace = FALSE)
null <- nnet::multinom(y ~ 1, data = dat, trace = FALSE)
1 - full$deviance / null$deviance # sample deviance R2, close to the .10 target

```

Example: a single predictor, R-squared = .10

```
## [1] 0.1071841
```

```

Sigma <- matrix(c(1, .3, .2,
                  .3, 1, .4,
                  .2, .4, 1), nrow = 3)

dat3 <- sample_multinomial_r2(n = 8000, R2 = .20, k = 3, y1 = 3,
                             beta = matrix(c(1, 0, 0, 1, 0, 0), nrow = 3, byrow = TRUE),
                             x_cov = Sigma, seed = 1)
full3 <- nnet::multinom(y ~ x1 + x2 + x3, data = dat3, trace = FALSE)
null3 <- nnet::multinom(y ~ 1, data = dat3, trace = FALSE)
1 - full3$deviance / null3$deviance # close to the .20 target

```

Example: three correlated predictors, R-squared = .20

```
## [1] 0.1958391
```

Ordinal model

The proportional-odds (ordinal) model shares a single linear predictor across all $y1 - 1$ cumulative logits, each with its own cutpoint; the baseline cutpoints below are equally spaced in log-odds, giving equally sized outcome categories when $\beta = 0$. As above, the population deviance R-squared is computed exactly on the quadrature grid, from the resulting category-probability matrix.

```

sample_ordinal_r2 <- function(n, R2, k = 1, y1 = 3, beta = rep(1, k),
                             intercept = 0, x_mean = rep(0, k), x_cov = diag(k),
                             order = 10, seed = NULL) {

  cutpoints0 <- intercept + stats::qlogis(seq_len(y1 - 1) / y1) # equally spaced baseline groups

  ## Gauss-Hermite quadrature grid over the k predictors (statmod::gauss.quad.prob()),
  ## transformed through the Cholesky factor of x_cov and shifted by x_mean.
  gh <- statmod::gauss.quad.prob(order, dist = "normal")

```

```

g <- as.matrix(expand.grid(rep(list(seq_len(order)), k)))
Z <- matrix(gh$nodes[g], ncol = k)
w <- apply(matrix(gh$weights[g], ncol = k), 1, prod)
Xgh <- sweep(Z %*% chol(x_cov), 2, x_mean, "+")

## Category probabilities from the common linear predictor and the cutpoints:
## a difference of cumulative logits for the interior categories.
ordinal_prob <- function(eta, cutpoints) {
  zeta <- matrix(cutpoints, nrow = length(eta), ncol = length(cutpoints), byrow = TRUE)
  cum <- stats::plogis(zeta - eta)
  p <- cbind(cum[, 1],
             cum[, -1, drop = FALSE] - cum[, -ncol(cum), drop = FALSE],
             1 - cum[, ncol(cum)])
  p <- pmax(p, .Machine$double.eps); p / rowSums(p)
}

## Population deviance R-squared, as a function of a coefficient scale `a`
## (beta = a * beta direction), computed exactly on the quadrature grid.
r2_of_a <- function(a) {
  eta <- drop(Xgh %*% (a * beta))
  p <- ordinal_prob(eta, cutpoints0)
  p0 <- colSums(p * w) # population marginal probabilities
  D1 <- -2 * sum(w * rowSums(p * log(p))) # model deviance
  D0 <- -2 * sum(p0 * log(p0)) # null-model deviance
  1 - D1 / D0
}

## ---- Root-find the coefficient scale that produces the target R2 -----
upper <- 1
while (r2_of_a(upper) < R2 && upper < 1e6) upper <- upper * 2
a <- stats::uniroot(function(a) r2_of_a(a) - R2, c(0, upper))$root
beta_final <- a * beta

## ---- Draw the actual finite sample from the calibrated population -----
if (!is.null(seed)) set.seed(seed)
X <- sweep(matrix(stats::rnorm(n * k), n, k) %*% chol(x_cov), 2, x_mean, "+")
colnames(X) <- paste0("x", seq_len(k))
eta <- drop(X %*% beta_final)
prob <- ordinal_prob(eta, cutpoints0)

u <- stats::runif(n) # draw a category per case
cum <- t(apply(prob, 1, cumsum))
y <- 1L + rowSums(u > cum)
y <- pmin(y, y1)

data.frame(y = ordered(y, levels = seq_len(y1)), X)
}

```

Code

```

dat <- sample_ordinal_r2(n = 5000, R2 = .10, seed = 1)
full <- ordinal::clm(y ~ x1, data = dat)
null <- ordinal::clm(y ~ 1, data = dat)

```

```
1 - (-2 * as.numeric(logLik(full))) / (-2 * as.numeric(logLik(null))) # close to the .10 target
```

Example: a single predictor, R-squared = .10

```
## [1] 0.106662
```

```
Sigma <- matrix(c(1, .3, .2,  
                 .3, 1, .4,  
                 .2, .4, 1), nrow = 3)
```

```
dat3 <- sample_ordinal_r2(n = 8000, R2 = .20, k = 3, beta = c(1, 0, 0), x_cov = Sigma, seed = 1)
```

```
full3 <- ordinal::clm(y ~ x1 + x2 + x3, data = dat3)
```

```
null3 <- ordinal::clm(y ~ 1, data = dat3)
```

```
1 - (-2 * as.numeric(logLik(full3))) / (-2 * as.numeric(logLik(null3))) # close to the .20 target
```

Example: three correlated predictors, R-squared = .20

```
## [1] 0.201551
```